

SpendPartition: Scalable Cross-Delegation Spending Control for Delegated Payments

SC6109 BYOP Proposal — v1.4 (23 Sep 2026) · Team of 4 · course project deadline 25 Oct 2026

Changes from v1.3: title restored to the v1.1 wording; related-work scopes verified against primary sources; preliminary results added.

1. Background and problem statement

On-chain payment authority is increasingly delegated to programs that spend without per-transaction approval: ERC-4337 / ERC-7579 session keys, ERC-7710 delegations, ERC-7715 granted permissions. A principal typically delegates to several such parties at once — research, procurement, subscriptions. Every deployed control we reviewed binds its counter to a single delegation, a single session, or a single account, never to a set of delegates under one principal budget.

Concretely: a principal is willing to risk 100 USDC this week, and grants Delegate A and Delegate B a cap of 80 each because either may need most of it. Every payment is individually within its cap; combined spend reaches 160, and no on-chain check rejects it. The naive fix is a static 50/50 split. That is safe but not work-conserving: a legal request of 80 from A is rejected while B's half sits unused. Dynamic reallocation removes the waste only if the principal is online and pays gas at every adjustment — which is what delegating was meant to avoid.

2. Why existing systems are insufficient

System	Scope of the counter
MetaMask Delegation Toolkit, 28 caveat enforcers (amount, period transfer, streaming, allowlist, time, call count)	per delegation
ERC-7579 SmartSessions policies (ERC20SpendingLimit, ValueLimit, UsageLimit, TimeFrame)	per session: state keyed by session ConfigId × token × sender
ERC-7579 SpendingLimitHook (rhinestonewtf/experimental-modules)	account-wide per token, fixed one-week period, ERC-20 transfer calls only; no per-delegate dimension
AIP (arXiv 2603.24775 §3.4), CapLease (arXiv 2608.01710)	off-chain; AIP states that aggregate spend enforcement belongs to the runtime, not the token

Both endpoints therefore exist already: strict per-delegate partitioning, and one account-wide cap. Not found in reviewed sources as of September 2026: a single on-chain enforcement point governing N independent delegates under one principal budget with a tunable guarantee/surplus split, together with a measurement of the resulting trade-off. We do not claim the concept of an aggregate budget as original; the design descends from classical shared-buffer allocation (complete partitioning, complete sharing, sharing with a minimum allocation).

3. What we build (MVP)

One parameterised Solidity contract. Delegate i holds a reserved allocation r_i ; the shared surplus is $S = B_G - \sum r_i$; the single knob is $\rho = \sum r_i / B_G$. $\rho = 1$ reproduces today's per-delegate caps, $\rho = 0$ an account-wide pool, $0 < \rho < 1$ a guaranteed floor plus contested surplus. A debit draws from the delegate's own reservation first and then from the surplus, atomically inside the payment path; a payment that would breach the aggregate budget reverts and no value moves. Budget windows roll over lazily by epoch tag — $O(1)$ on first touch, no keeper, no iteration over delegates. Explicitly out of scope: MPC custody, multi-chain, ZK/FHE, a live merchant network, dispute arbitration, and borrowing-with-recall. The last is a finding rather than an omission: once value has left the contract it cannot be preempted.

4. Evaluation

Safety — property, invariant and fuzz tests that per-window $\sum \text{spend} \leq B_G$ under all schedules; differential testing against an independent reference implementation; mutation testing. *Isolation* — a request within a delegate's own unspent reservation is never rejected by the budget check, regardless of what other delegates have spent. *Utilization and fairness* — served demand and rejection distribution as ρ sweeps $0 \rightarrow 1$ under honest, aggressive and adversarial-ordering workloads, reported as measured curves with no shape asserted in advance. *Cost* — per-payment gas and deployment gas as N grows, reported as two curves.

5. Preliminary results

A working skeleton already runs (Foundry, solc 0.8.30): 29 tests pass, including the ordering and isolation scenarios above, with six configurations each driven by invariant campaigns of 8,192 calls. Measured on a local chain with every payment sent as its own transaction: a payment costs 47,123 gas at $N = 2$ and the same at $N = 50$, while deployment grows by about 24,971 gas per registered delegate. The N-dependent work is moved into one-time initialisation rather than removed. The attached figure shows both curves; logs and a one-command reproduction script are available.

6. Ten-minute demo

Six acts: over-granted caps overshoot the principal's intent; a static split blocks the overshoot but rejects a legal single-delegate demand; a shared pool serves that demand; an aggressive delegate starves an honest one; reservation restores the honest delegate's guarantee; a live ρ slider with gas and utilization readouts.

7. Course alignment, and difference from Options 1-6

Smart contracts, authorization and settlement semantics, and a scalability question stated as gas, storage growth and rejection behaviour under growing N and adversarial ordering — not TPS. Option 1 is an intent execution network, 2 a payment appchain, 3 and 4 DA/rollup benchmarking, 5 a parallel EVM executor, 6 a cross-rollup router. None addresses what a principal's aggregate authorization actually permits.

8. Team of four, and cost

A contracts and interfaces · **B** verification (property, invariant, differential, mutation) · **C** adversarial harness and demo driver · **D** measurement, dashboard, report and video. Semantics, property tests and storage layout were frozen before implementation and the ABI is fixed, so the four streams run in parallel. Cost: zero — Foundry and a local chain; public testnet deployment is optional and served by free faucets.